

# CIFRADO DE UNA SOLA VÍA

Ciencias de la Computación II

Universidad Distrital Francisco José de Caldas

2026

Sofía Rivera

Juan Esteban Ávila Trujillo

Luis Fernando Lopez Pardo

Juan Esteban Moreno Durán

Bogotá, Colombia

24 de Junio de 2026

# Introducción

El presente documento analiza e implementa un algoritmo asimétrico para la encriptación de mensajes tomando como base el programa de generación pseudoaleatoria de números, esto resulta fundamental para entender conceptos claves dentro de la ciberseguridad actualmente y como esta se va haciendo más robusta y sofisticada con el tiempo.

# Índice

|                                                      |          |
|------------------------------------------------------|----------|
| <b>Introducción</b>                                  | <b>1</b> |
| <b>1. Marco teórico</b>                              | <b>3</b> |
| 1.1. Algoritmos asimétricos . . . . .                | 3        |
| 1.2. Hashing . . . . .                               | 3        |
| <b>2. Generador de números pseudoaleatorio</b>       | <b>3</b> |
| 2.1. ¿Por qué funciona para este contexto? . . . . . | 3        |
| <b>3. Cifrado</b>                                    | <b>3</b> |
| 3.1. Implementación HASH . . . . .                   | 3        |
| 3.2. ¿Qué cambió respecto al generador? . . . . .    | 4        |
| 3.3. Rol de cada componente . . . . .                | 4        |
| 3.4. La sal: protección adicional . . . . .          | 4        |
| 3.5. Flujo del algoritmo . . . . .                   | 4        |
| 3.6. Ataque de diccionario . . . . .                 | 5        |
| <b>4. Conclusiones</b>                               | <b>5</b> |

# 1. Marco teórico

## 1.1. Algoritmos asimétricos

Esquema criptográfico que emplea dos claves relacionadas matemáticamente: una pública, compartible libremente, y una privada, conocida solo por su propietario. Lo que una cifra, solo la otra puede descifrar. Su seguridad se basa en problemas matemáticamente irreversibles como la factorización de números primos grandes (RSA) o las curvas elípticas (ECC), operaciones triviales en una dirección e inviables en la contraria.

## 1.2. Hashing

Un hash es una función que transforma una entrada de cualquier tamaño en un número de tamaño fijo. A diferencia del cifrado, el hashing es **irreversible**: no existe operación que permita recuperar el mensaje original a partir del hash resultante. Su propósito no es ocultar y recuperar información, sino **verificar** que una entrada produce el mismo resultado esperado.

# 2. Generador de números pseudoaleatorio

## 2.1. ¿Por qué funciona para este contexto?

El generador de números aleatorios se desarrolló con una herramienta que resultó muy útil, la siguiente función:

$$x = (a \cdot x + c + n \cdot w) \text{ mód } m$$

Donde:

$a$  = Multiplicador;  $x$  = Estado actual (inicia en la semilla);  $c$  = Incremento;  $n$  = Número de iteración;  $w$  = Peso por iteración;  $m$  = Módulo primo

- **Uniformidad e independencia:** Gracias a la función el programa puede parecer que genera resultados aleatorios cuando en realidad es determinista, por lo que la misma entrada **SIEMPRE** va a producir la misma salida.
- **Asimetría:** El resultado de la operación es imposible o computacionalmente inviable de revertir, por dos características principales:
  - **Módulo primo:** al aplicar la operación módulo se pierde información permanentemente, ya que infinitos valores distintos pueden producir el mismo residuo.
  - **Cada resultado depende del anterior:** el estado se acumula de forma encadenada, por lo que no es posible reconstruir un eslabón sin conocer todos los previos.

# 3. Cifrado

## 3.1. Implementación HASH

Como ya se dijo anteriormente, la base del programa fue el generador de números pseudoaleatorios, y en especial se tomó su función transformándola para nuestro caso de hashing. La función implementada es la siguiente:

$$x = ((a \cdot x) \oplus \text{valorLetra}) \bmod m$$

Donde:

$a$  = Multiplicador (31, primo);  $x$  = Estado acumulado (inicia en la semilla 3);  $\text{valorLetra}$  = Valor ASCII del carácter actual;  $m$  = Módulo primo (1 000 000 007)

### 3.2. ¿Qué cambió respecto al generador?

El generador original usaba sumas para combinar los términos, mientras que la implementación hash introduce la operación **XOR** ( $\oplus$ ), la cual mezcla los bits del estado acumulado con los del carácter de forma no lineal. Este cambio hace que la relación entre entrada y salida sea mucho más difícil de predecir.

### 3.3. Rol de cada componente

- **Multiplicador primo** ( $a = 31$ ): Antes de mezclar cada carácter, el estado  $x$  se multiplica por 31. Esto *dispersa* el estado anterior por el espacio numérico, garantizando que dos estados cercanos se alejen significativamente antes de la siguiente operación. Si se usara un valor como 32 (potencia de 2), la multiplicación equivaldría a un simple desplazamiento de bits, produciendo resultados predecibles.
- **XOR** ( $\oplus$ ): Mezcla el estado dispersado con el valor del carácter actual. Es reversible solo si se conoce uno de los operandos, pero como  $x$  ya fue transformado por el módulo en iteraciones previas, esa información se perdió.
- **Módulo primo** ( $m = 1\,000\,000\,007$ ): Limita el resultado al rango  $[0, m-1]$  y destruye información en cada iteración. Al ser primo, no posee estructura interna de factores que agrupe resultados de forma sistemática, distribuyendo los hashes de manera uniforme y reduciendo la probabilidad de **colisión** (dos entradas distintas con el mismo hash).

### 3.4. La sal: protección adicional

El programa concatena una cadena fija (sal) al mensaje del usuario antes de aplicar el hash:

$$\text{"HOLA"} + \text{"Z9xY"} \longrightarrow \text{"HOLAZ9xY"}$$

La sal **no altera el mensaje original**, solo extiende la entrada al algoritmo. Su propósito es impedir ataques con tablas precalculadas (Rainbow Tables), ya que el atacante necesitaría reconstruir toda su tabla para cada sal distinta, volviendo el ataque computacionalmente inviable.

### 3.5. Flujo del algoritmo

La Figura 1 ilustra el recorrido completo del mensaje desde la entrada del usuario hasta la verificación mediante ataque de diccionario.

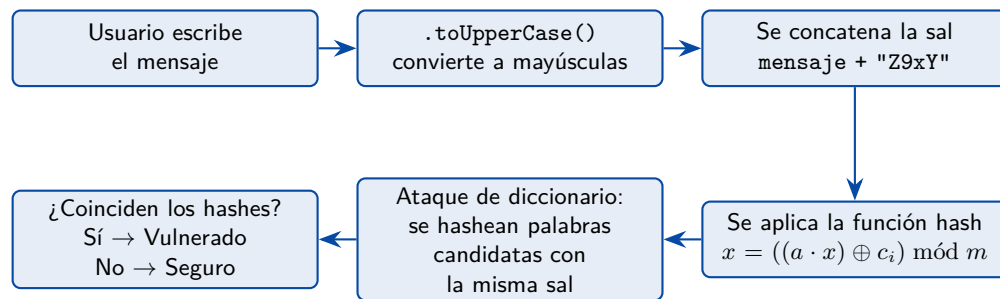


Figura 1: Flujo completo del algoritmo: desde la entrada del usuario hasta la verificación por diccionario.

### 3.6. Ataque de diccionario

Para demostrar la importancia de contraseñas robustas, el programa incluye una simulación de ataque de diccionario. Se define un arreglo con palabras comunes y predecibles; el programa calcula el hash de cada una utilizando la misma sal y lo compara con el hash del mensaje original. Si alguno coincide, la palabra queda expuesta. Esto evidencia que el algoritmo hash **no necesita ser revertido** para ser vulnerado: basta con probar candidatos y comparar resultados. Una contraseña fuerte es aquella que no aparece en ningún diccionario previsible del atacante.

## 4. Conclusiones

La implementación presentada demuestra cómo a partir de un generador congruencial lineal se puede construir una función hash funcional, aplicando operaciones matemáticas que garantizan irreversibilidad: el módulo primo destruye información en cada iteración y el XOR introduce no linealidad en la mezcla de caracteres. La elección de constantes primas, tanto para el multiplicador como para el módulo, resulta fundamental para distribuir los resultados de forma uniforme y minimizar colisiones. Complementariamente, mecanismos como la sal protegen contra ataques con tablas precalculadas, mientras que la simulación de ataque de diccionario evidencia que la verdadera fortaleza de un sistema de hashing depende tanto del algoritmo como de la complejidad de la entrada elegida por el usuario.